PROGRAMMING 622 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Mohamed Elmudathir Ibrahim Ahmed

Student ITS No: 402414439

Qualification: BSC in IT Year of Study: 2 Semester: 2

Assignment due date: 21/09/2026 Date submitted: 20/09/2026

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents Introduction

Main Body (with relevant subheadings) Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: muddathir Date: 20/09/2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

How to Run

1. Extract the files using 7-zip winrar or windows built in extractor etc.

2. Add all the files as a project (only .cpp files)

3. Once added, go to main.cpp, click on Build (the gear icon at the top then click on the play button to run it, or you could just click Build and run if you don't want to build it and then run it separately.

4. Once you run the program, enter a username and password when prompted to log in as an engineer (see the testing credentials below).

5. The program then runs the whole simulation automatically, everything from the sensor data, city logs, event processing, emergency handling, system reports, and file persistence, displaying the output to the console as it goes.
   After running, the program will have created the following files in the same folder: engineers.dat, events.dat, city_logs.dat, config.txt, and city_logs_export.csv.

6. To exit the program, you simply need to press any eg. the Enter key.

Testing Login Credentials

- Username: Mohamed, Password: money$$$1 Clearance: High
- Username: Jared, Password: bankai1, Clearance: Medium
- Username: Shaun, Password: poji325, Clearance: Low

These are the containers I used, and also why I used them.

vector<Engineer> stores the Engineer objects in a vector so search for valid credentials by traversing through the vector is fast and simple, matching the STL use the assignment requires.

vector<double> (sensor readings) — a vector gives O(1) random access by index, which suits sensor readings that may need to be looked up, sorted, or scanned repeatedly.

list<string> (historical city logs) — a linked list gives O(1) insertion (no shifting of other elements), which suits a log that keeps growing indefinitely, since new entries are constantly being appended and older entries are rarely accessed by position.

queue<Event> — regular city events are processed strictly in the order they arrive (FIFO), so a queue is used. This guarantees older events are never indefinitely delayed by newer ones arriving.

stack<EmergencyEvent> — emergencies are processed most-recent-first (LIFO), since a newly occurring emergency is likely still escalating and needs immediate attention, even while an earlier one is already being handled. A stack models this directly.

File Persistence

engineers.dat, events.dat, and city_logs.dat are written and read back using ofstream/ifstream in binary mode, one record's fields per line, and reconstructed back into real Engineer/Event/string objects on load. config.txt is a plain text configuration file (system name, version, max engineers). city_logs_export.csv is a plain-text export of the historical city logs, one entry per line.

Note: queue does not support iterators (begin()/end()), so <algorithm> functions like count_if cannot run directly on it. A separate vector<Event> (eventsv) is kept alongside the queue specifically so <algorithm> functions have something iterable to work with, while the queue itself still handles FIFO processing as required.

Big-O Focus

- Analyse login search complexity (linear vs binary search):

Linear search is a type of algorithm that searches data in any order; the bigger the dataset, the slower it gets, which is why its Big-O notation is O(n), and why it's better suited for smaller datasets. Binary search is an algorithm that searches data that must be in sorted order. The bigger the dataset, the better it gets because the data is ordered, so it's easier to find the correct value. Its big O notation is O(log2n), and it's better suited for bigger datasets. You would choose linear over binary specifically in this case because it doesn't matter where or when the engineers were added; also, keeping the vector sorted (which binary search would require) means extra work on every insert, and for the small number of engineers in this system, that extra work isn't worth it. Binary search would actually be the better choice when dealing with large datasets because the dataset being ordered makes finding the value easier and quicker.

Linked lists are best suited for the logs because you don't have to rearrange elements, unlike a vector; when adding things, they have to be shifted, but with linked lists you can insert them without any cost.

Big-O Analysis

- Compare insertion and traversal complexities

So traversal, especially traversing a vector, requires the vector to contain elements you can traverse an empty vector: .begin() and .end() point to the same place (since there's nothing between "start" and "one past the end" when the vector is empty), so the loop's condition it != end() is false immediately, and the loop body just never runs. You can use .begin and .end, and an iterator, e.g., it++, if you want to remove an element; it requires the position of the element, then after it erases it, it shifts every element after it to fill the space. For list traversing, it requires an iterator to point to the current node to preserve the integrity of the list and also ensures we don't lose the position when traversing. Also, it points to the address of the next node in the list. Removing a value from the list requires the actual value after you give it the value it searches for, it removes the node linked by the pointers. To conclude, traversal for both vectors and lists in big O notation is O(n), and vector removal is O(n). For

list removal, the finding of the actual value is O(n), but the actual removal step once found is written as O(1).

Big-O Justification: Queue vs Stack

- Explain why stack vs queue is appropriate

FIFO (queue) is the right choice for regular city events because it guarantees every event is handled strictly in the order it arrived, preventing older events from being delayed indefinitely by newer ones constantly jumping ahead. LIFO (stack) is the right choice for emergencies because the most recently occurring emergency is addressed first, since a newly occurring emergency is likely still escalating and needs immediate attention, even while an earlier emergency is already being handled.

Big-O justification

- Algorithms

Sort: Sort has a time complexity of O(n log n) because the algorithm repeatedly splits the data in half, which takes log n splits (as shown by the halving pattern — 8 elements takes 3 splits, similar to binary search). At each of those splitting levels, the algorithm still needs to compare and organize all n elements remaining — that's On work per level. Multiplying the number of levels by the work per level gives the overall complexity of n log n. Example: 8 elements [8,7,6,5,4,3,2,1]=

1st split: [8,7,6,5] [4,3,2,1]

2nd split: [8,7] [6,5] [4,3] [2,1]

3rd split: [8],[7],[6],[5],[4],[3],[2],[1]

So $\log_2(8) = 3$

Min_element is also O(n) because if it doesn't check every element, there is no way the algorithm can know it has found the minimum value; the same is true for max_element: without checking, how can it be sure it has found the maximum element. Count_if is O(n) because, in the worst case, it has to check every single element to match the given condition.

find() is O(n) because, in the worst-case scenario, it's the very last element or it's not found in the vector at all.

Worked examples (using the sensor readings vector after the first element is removed and the remaining values are sorted: (7.0, 9.0):

find(9.0): check index 0 ($7.0 \neq 9.0$), check index 1 ($9.0 = 9.0$) → found after 2 checks. In the worst case (the value is the last element, or not present at all), every element must be checked so its O(n). Min_element(): start with 7.0 as the current minimum, compare against 9.0 (9.0 is not smaller, so the minimum stays 7.0) → both elements had to be checked before it could be certain that 7.0 is the true minimum — O(n).

max_element(): start with 7.0 as the current maximum, compare against 9.0 (9.0 is larger, so the maximum updates to 9.0) → both elements had to be checked before it could be certain 9.0 is the true maximum — O(n).

count_if(isFireEvent) on the emergency report vector [flood, fire, fire, earthquake, fire]: check “flood” (no match), “fire” (match, count = 1), “fire” (match, count = 2), “earthquake” (no match), “fire” (match, count = 3) → all 5 elements had to be checked to reach the final count of 3, so it's O(n).